MCP and GraphRAG

Open tool access and graph-grounded answers for Portage and Sora

Same tool registry · Propose, then approve · Grounded in live data

Portage v0.1 · Document date: 12 October 2026

Audience: engineers, product owners, security reviewers

Status: Delivered and tested offline (111 tests).

Verified with the MCP Python SDK client over stdio; not yet tried in VS Code or Claude Desktop.

0 Contents

1 Overview and goals

2 Decisions at a glance

3 GraphRAG

4 MCP server

5 Safety and security model

6 Testing and validation

7 Operations and troubleshooting

8 Limitations and roadmap

1 Overview and goals

Two capabilities extend Sora's tool registry. **GraphRAG** lets Sora answer questions about how things relate (which Rally fields feed a Jira field, which mapping version changed it, which catalog a failure points at) by retrieving a connected neighbourhood of Portage's own data instead of isolated lookups. The **MCP server** exposes the same governed registry to any Model Context Protocol client, so other agents can read Portage and propose changes, with the same human approval.

Grounded

Answers come from the live graph and documentation, not from model memory.

Current

The graph is rebuilt from the store on every call.

Governed

Read tools only read; write tools only propose.

Open

One registry serves Sora and external MCP agents.

2 Decisions at a glance

Topic	Decision	Why
Graph construction	Built deterministically from the store and rebuilt on each call.	Exact and always current. No extraction errors from a small local model, and no index to keep in sync.
Retrieval	Token match, multi-hop traversal and full-text documentation search.	No embeddings or vector database to run; everything stays local.
Graph storage	In-memory adjacency lists; no graph database or library.	Few dependencies, in line with the rest of Sora. Cheap at the current data size.
Where it lives	Two read tools, `graph_search` and `graph_overview`, in the shared registry.	Sora and MCP clients get them with identical governance.
MCP transport	stdio only.	Portage has no authentication; stdio keeps the trust boundary at the local user.
MCP approvals	No apply tool.	A person approves in Portage; an agent cannot approve its own proposal.
MCP validation	The registry validates; the SDK's strict schema pre-check is off.	The registry tolerates string-encoded arguments and returns corrective errors.
MCP SDK	`mcp>=1.9,<2`	The 2.x release changed the server API.
Settings	One typed registry. A saved value wins over an environment variable, which wins over the default. The Settings page and API are generated from it.	Adding a setting is one definition, and every setting is validated, audited and resettable.

3 GraphRAG

Classic RAG retrieves text passages. Many Portage questions are about connections, so Sora retrieves a *subgraph*: the items a question names, plus the relationships around them, plus the documentation that explains those kinds of item. Because Portage's data is structured, the graph is derived directly from it rather than extracted from text by a model.

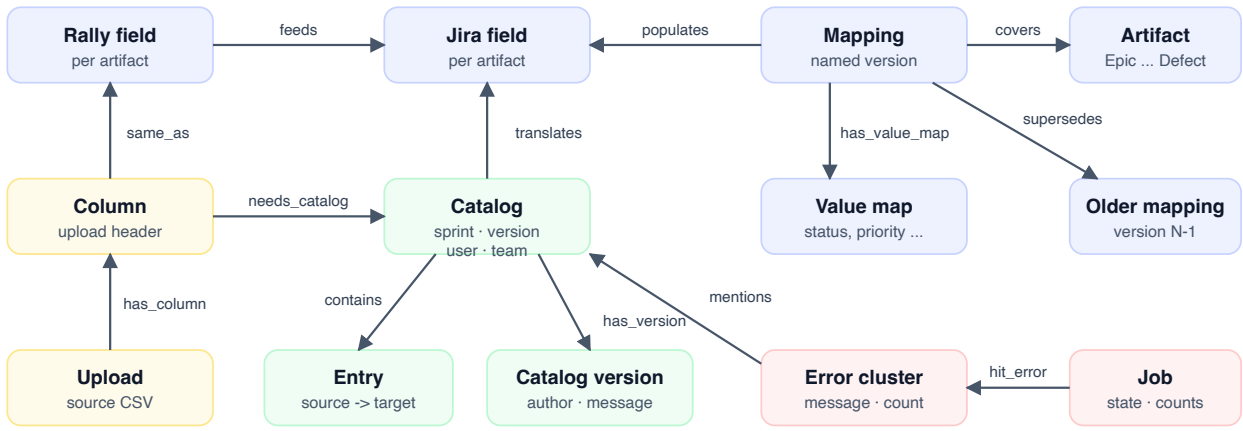


Figure 1. Graph schema. Indigo: mapping lineage. Green: catalogs. Amber: uploads. Red: jobs. Error clusters also link to Jira fields they mention.

Node types

Type	Built from	Key properties
`artifact`	Field catalog artifact types	Epic, Feature, User Story, Task, Defect
`mapping`	Saved mapping versions	name, version, description
`jira_field`	Mapping field maps, per artifact	key, artifact, data type
`rally_field`	Mapping sources, per artifact	key, artifact
`valuemap`	Mapping value maps	field, number of values, sample pairs
`catalog`	Sprint, version, user and team catalogs	kind, entry count
`entry`	Catalog entries	source, target
`catalog_version`	Catalog history (latest 6)	version, author, message, time
`source`	Uploaded CSVs (latest 8, header only)	id, column count
`column`	CSV header names	name
`job`	Persisted migration jobs (latest 10)	state, totals, created, failed
`error`	Failed items, digits collapsed (top 5 per job)	message, occurrences

Relationships

Relationship	Connects	Meaning
`covers`	Mapping to Artifact	A versioned name (jira_us_mapping_vN) covers only its own artifact.
`populates`	Mapping to Jira field	Carries the Rally source columns and the transform.
`feeds`	Rally field to Jira field	Lists every mapping version that uses the link: the lineage edge.

Relationship	Connects	Meaning
`supersedes`	Mapping to older mapping	Newer version replaces the previous one of the same artifact.
`has_value_map`	Mapping to Value map	Status, priority and similar value translations.
`translates`	Catalog to Jira field	sprint to sprint; version to fix_versions; user to assignee, reporter; team to team.
`contains`	Catalog to Entry	One node per Rally value and its Jira value.
`has_version`	Catalog to Catalog version	History with author (user or copilot) and message.
`has_column`	Upload to Column	Header of an uploaded Rally CSV.
`same_as`	Column to Rally field	Case-insensitive name match.
`needs_catalog`	Column to Catalog	Iteration or Sprint to sprint; Release, Milestone, Fix Version to version; Owner, Submitted By to user; Team to team.
`hit_error`	Job to Error cluster	Failed items grouped by message with numbers collapsed (Sprint 77 and Sprint 78 become one cluster).
`mentions`	Error cluster to Catalog or Jira field	Linked when the message names the catalog kind or field.

Retrieval pipeline



Scoring. A question term scores 3 times its rarity weight when it appears in a node's label, and 1 times when it appears only in its properties, where rarity is log(1 + nodes / nodes containing the term) over the graph. A node whose whole label appears in the question gets +5. Error clusters get +6 when the question is about failure ("fail", "error", "missing", "unmapped"), so "why did it fail" lands on the cause rather than the generic field.

Parameter	Value
Seeds	Up to 4 best matches, each scoring at least 60% of the best.
Hops	1 to 3, default 2 (GraphRAG setting).
Fan-out	5 neighbours per relation per node.
Bulk relations	populates, contains and has_column are followed only toward nodes that match the question.
Node cap	24 nodes per answer (GraphRAG setting, 8 to 60).
Relationships	Closest to the seeds first, at most 30.
Documentation	Up to 3 passages: the top 2 full-text hits plus the glossary entries for the kinds of node found, 300 characters each.
Result budget	5,500 characters (the agent truncates tool results at 6,000). The tail is trimmed and the answer is flagged as trimmed.

Worked examples

Real output from the test workspace: two User Story mapping versions, one sprint catalog with two entries, one uploaded CSV and one finished job with two failed items.

Lineage question.

```
question: which Rally fields feed the sprint field for User Story

seeds
  jira_field:sprint (User Story)
  artifact:User Story

relationships (top of the ranked list)
  rally_field:Iteration (User Story) --feeds--> jira_field:sprint (User Story)
      [via jira_us_mapping_v1, jira_us_mapping_v2]
  catalog:Sprint catalog --translates--> jira_field:sprint (User Story)
  error:Sprint # not found in the sprint catalog --mentions--> jira_field:sprint (User Story)
  mapping:jira_us_mapping_v1 --populates--> jira_field:sprint (User Story) [from Iteration]
  mapping:jira_us_mapping_v2 --populates--> jira_field:sprint (User Story) [from Iteration]
  mapping:jira_us_mapping_v2 --supersedes--> mapping:jira_us_mapping_v1

docs: Mapping Studio, Jira Catalog, Value maps
```

Failure question. The two failures (Sprint 77 and Sprint 78) collapse into one cluster, which points at the sprint catalog and the Jira sprint field, and the graph shows that the catalog's two entries are Sprint 24 and Sprint 25.

```
question: why did my sprint migration fail

seeds
  error:Sprint # not found in the sprint catalog [occurrences=2]

relationships
  error --mentions--> catalog:Sprint catalog
  error --mentions--> jira_field:sprint (User Story)
  job:job job1 (done) --hit_error--> error:Sprint # not found in the sprint catalog
  catalog:Sprint catalog --contains--> entry:Sprint 24 -> 1042
  catalog:Sprint catalog --contains--> entry:Sprint 25 -> 1043
  catalog:Sprint catalog --has_version--> catalog_version:Sprint catalog v2
  column:Iteration --needs_catalog--> catalog:Sprint catalog
  column:Iteration --same_as--> rally_field:Iteration (User Story)

docs: Jira Catalog, Migration
```

Global view: graph_overview

Broad questions such as "what is the state of my workspace?" get a summary per community instead of a neighbourhood: mapping versions per artifact, catalogs with their latest change, job states and the most frequent errors.


```
{
 "artifacts": [{"artifact": "User Story", "mapping_versions": 2,
                "latest": "jira_us_mapping_v2", "mapped_fields_in_latest": 13}],
 "catalogs":  [{"catalog": "Sprint catalog", "kind": "sprint", "entries": 2, "versions": 2,
                "last_change_by": "copilot", "last_change": "Copilot: upsert 1 entries"}],
 "jobs":      {"count": 1, "by_state": {"done": 1},
               "top_errors": [{"error": "Sprint # not found in the sprint catalog",
                               "occurrences": 2, "related": ["sprint (User Story)", "Sprint catalog"]}]},
 "uploads": 1
}
```


Tools and skill

Item	What it does
`graph_search`	Parameters: `query` and `depth` (1 to 3, default 2). Returns seeds, relationships, nearby nodes, documentation and statistics.

Item	What it does
`graph_overview`	No parameters. Returns per-artifact, per-catalog and per-job summaries.
`@lineage`	Lineage Analyst skill with graph_search, graph_overview, list_mappings, get_mapping, compare_mappings and search_docs. Routed by words such as lineage, impact, trace, depends on, relationship and which mappings, or by mentioning it explicitly.
Suggested prompts	Which Rally fields feed the Jira sprint field for User Story? What is the state of my workspace? Which catalogs and fields are affected by the failures in my last job?

4 MCP server

The Model Context Protocol lets an AI client discover and call tools from a server. Portage's server publishes the existing tool registry, so there is one implementation of every tool and one set of rules. It runs over stdio: the client starts it as a subprocess.

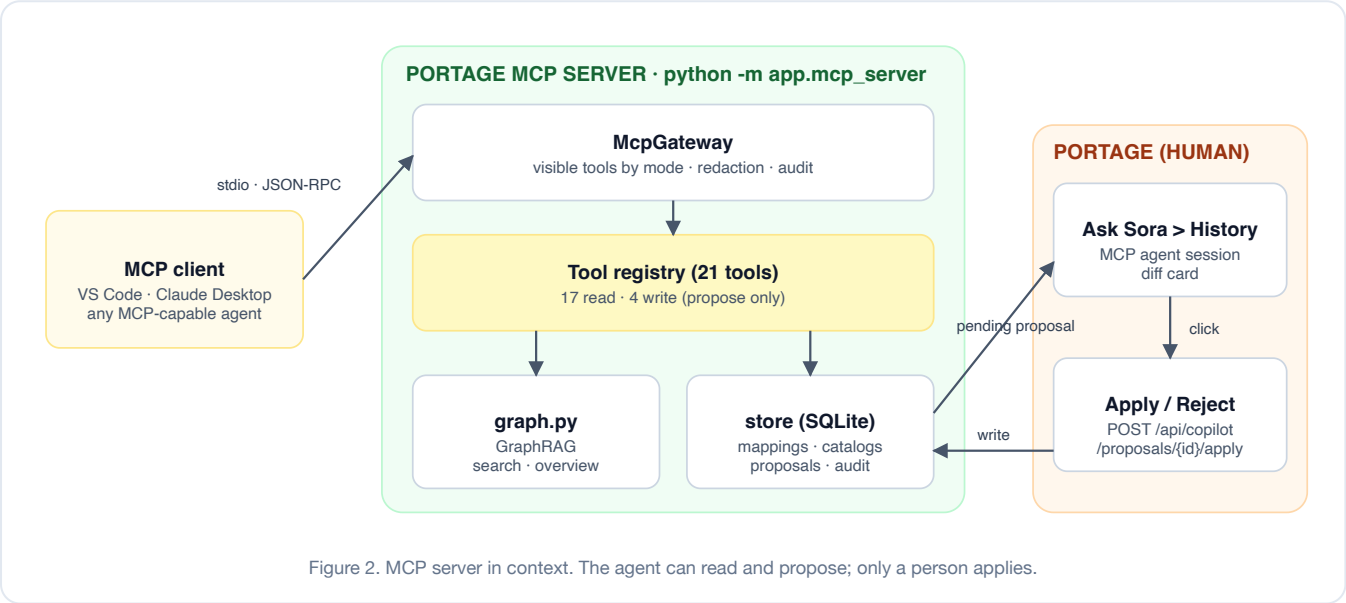


Figure 2. MCP server in context. The agent can read and propose; only a person applies.

Life of a write request

Agent calls propose_catalog_upsert over stdio ▸ **Gateway** mode check, restore placeholders ▸ **Registry** validate, diff, store proposal ▸ **Proposal** pending, shown in History ▸ **Person** reviews the diff, clicks Apply ▸ **Apply** re-derive, write, audit

Governance

Control	Behaviour
Propose only	Write tools create a pending proposal and write nothing. No apply tool is exposed; the result tells the agent that a person must apply it in Portage.
Read-only mode	Sora's read-only mode, or the MCP read-only setting, removes every `propose_*` tool from the tool list and refuses calls to them. MCP is never more permissive than Sora.
Redaction	MCP has its own redaction setting, on by default and independent of Sora's. It masks emails as <email_N> and secrets one way in results. Placeholders in arguments are restored before validation, so the reviewer sees real values.
Access switch	Settings, MCP server can turn MCP off or hide individual tools. Checked on every call, so a connected client loses access at once.
Audit	Each server process opens a conversation `mcp-<8 hex>`. Every call is recorded with via=mcp, and proposals appear in History as an MCP agent session.

Control	Behaviour
Validation	Arguments are checked by the registry's Pydantic models, which tolerate JSON-string arguments and "null". Problems come back as error results so the agent can correct itself.
Unknown tools	Names that do not exist, or are hidden by the mode, return an error result.
Tool hints	Read tools are annotated read-only. Write tools are annotated not read-only and not destructive, because they only propose.
Responsiveness	Calls run on a worker thread so the protocol loop stays responsive.

Settings

Everything an administrator can tune lives on the Settings page (sections: Sora assistant, MCP server, GraphRAG, System) and in the API at `/api/settings`. Changes are validated, audited and take effect without a restart.

Setting	Default	Effect
MCP: Allow MCP access	on	Master switch; when off, clients see no tools.
MCP: Read-only	off	Hides every propose_* tool. Always on while Sora is read-only.
MCP: Mask emails and secrets	on	Redaction for results sent to MCP clients, independent of Sora.
MCP: Hidden tools	none	Tools MCP clients cannot see or call.
GraphRAG: Default depth	2	Hops used when a caller gives none (1 to 3).
GraphRAG: Max items per answer	24	Upper bound on nodes in one answer (8 to 60).
Sora: model, URL, mode, redaction, steps	see Sora document	Same store as the assistant's own panel.

The Settings page also shows ready-to-paste client configuration for the installation. No secret can be stored in settings.

Running it

From the repository root:

```
PYTHONPATH=backend PORTAGE_DATA_DIR=backend/data .venv/bin/python -m app.mcp_server
```

VS Code (shipped as `.vscode/mcp.json`; start the portage server from the MCP list):

```
{
  "servers": {
    "portage": {
      "type": "stdio",
      "command": "${workspaceFolder}/.venv/bin/python",
      "args": ["-m", "app.mcp_server"],
      "env": {
        "PYTHONPATH": "${workspaceFolder}/backend",
        "PORTAGE_DATA_DIR": "${workspaceFolder}/backend/data"
      }
    }
  }
}
```

Other clients use the same command in their own configuration file, for example (replace the paths):

```
{
  "mcpServers": {
    "portage": {
      "command": "/absolute/path/to/portage/.venv/bin/python",
      "args": ["-m", "app.mcp_server"],
      "env": {
        "PYTHONPATH": "/absolute/path/to/portage/backend",
        "PORTAGE_DATA_DIR": "/absolute/path/to/portage/backend/data"
      }
    }
  }
}
```

5 Safety and security model

Risk	Control
An external agent changes data	It cannot: the only write path is a proposal, and applying needs a person in Portage. Enforced by the server, not by instructions.
Data leaves the machine	Sora's model is local, but an MCP client may be a cloud model. Anything a tool returns then goes to that provider. Emails and secrets are masked; person names are not. Use a local client for sensitive data.
Credentials exposed	No tool reads Jira or Rally tokens, and none can set one. connection_status returns booleans, base URL and project key only.
Prompt injection from data	Catalog entries, error messages and CSV values are returned as data. No tool sends data elsewhere, and delete and run-migration tools do not exist.
Unauthorised access	stdio only: the server is started by, and talks to, the local user's MCP client. There is no network listener. Portage itself still has no user authentication.
Graph poisoning	A hostile string in a catalog or job message could appear in an answer, but it can only be returned as text; it cannot trigger a write.
Runaway output	Graph answers are capped at 24 nodes, 30 relationships and 5,500 characters.

Know where the data goes. Sora runs on a local model, so nothing leaves the machine. An MCP client is a different matter: if it uses a cloud model, every tool result is sent to that provider. Redaction masks emails and secrets, not person names. Connect cloud-backed clients only to workspaces where that is acceptable.

6 Testing and validation

111
offline tests, all passing (78 before, 33 added)

7
GraphRAG tests

7
MCP tests, one over a real stdio session

19
settings tests

File	Tests	Covers
`test_graph.py`	7	Lineage, version chain and catalog links; error clustering and linking to catalogs and fields; multi-hop search with docs inside the size budget; failure questions landing on the error cluster; no-match safety; overview; graph tools are read-only with bounded depth.
`test_mcp.py`	7	All registry tools exposed with schemas and hints; propose-only behaviour, with the proposal visible in History and applied through the Portage API; read-only mode; errors returned, not raised; email masking and restoration; audit trail; a real stdio session (initialize, list tools, call tools).

File	Tests	Covers
`test_settings.py`	19	Saved over environment over default, reset, validation and clamping, settings saved by the previous version, audit of changes, MCP never more permissive than Sora, hidden tools, access switch, independent MCP redaction, graph settings driving graph_search, the API and the shared Sora store, and connection snippets.

Defects found and fixed. The first answers exceeded the agent's 6,000-character limit, so results are now trimmed to a budget. Relationships were capped in insertion order, so mapping edges crowded out the error links, and bulk relations pulled in arbitrary fields; both were fixed by ranking relationships by closeness to the seeds and following bulk relations only toward matches. Reviewing the example output then showed weak side matches (fields matching only the "User Story" suffix) in the seed list, fixed with a relative score threshold.

```
cd backend && pytest tests --ignore=tests/test_live_ollama.py
```

7 Operations and troubleshooting

Symptom	Check
The server does not appear in VS Code	Check that `.venv/bin/python` exists, then run the command from section 4 in a terminal. VS Code's MCP output shows its stderr.
No module named mcp	Install it in the same environment: `pip install "mcp>=1.9,<2"` . The 2.x SDK changed its server API and is not supported.
Fewer tools than expected	Check Settings, MCP server: Read-only, Hidden tools and Allow MCP access. MCP is also read-only whenever Sora's mode is read-only.
A proposal is not visible in Portage	Open Ask Sora, History, and look for the MCP agent session. Make sure the server and the backend use the same `PORTAGE_DATA_DIR` .
Different data from the UI	The database lives in ./data relative to where the process starts. Pin it with `PORTAGE_DATA_DIR` .
Protocol errors	stdout carries the protocol. The server logs to stderr only; do not print to stdout.
A graph answer misses an item	The graph covers the latest 8 uploads, 10 jobs and 6 catalog versions per catalog. Call graph_overview, then repeat with exact names.

8 Limitations and roadmap

Current limitations

- The graph is rebuilt on every call. This is fine at current sizes; add caching and invalidation if data grows.
- Matching is token based. A paraphrase that shares no words with a label or property can miss; there are no embeddings.
- Job records do not store which mapping or catalogs a run used, so the graph has no job-to-mapping link.
- Coverage is bounded: latest 8 uploads, 10 jobs, 6 catalog versions per catalog.
- MCP is stdio only, one session per server process, and there is no per-client identity or permission.
- The MCP SDK is pinned to 1.x.

Next steps

Area	Scope
Retrieval	Hybrid retrieval with embeddings; community summaries for very large workspaces; cached or incremental graph; record mapping and catalog ids on jobs to add job lineage.
MCP	Streamable HTTP with authentication and per-client identity; read-only clients; MCP resources and prompts; migration to the 2.x SDK.
Quality	Golden-question evaluation for graph answers in CI; telemetry on tool use and proposal acceptance across Sora and MCP.